Applied Accelerated AI — Weekly Practice Tests

IIT Guwahati / NPTEL · Weeks 1–12 · 177 questions

Each week has the questions first, then an answer key with explanations on the following page. Cover the key, attempt the questions, then check. Questions mix official 2026 assignment content (some reworded), new scenario and calculation questions, and items marked **(2022)** from the older NPTEL quiz screenshots in the repo.

Week 5 has no material in the repository; its test is a gap-filler built from the course outline (TensorFlow acceleration) and is not official.

Week	Topic	Qs
Week 1	AI Hardware & Accelerators	13
Week 2	Containerized AI Systems	14
Week 3	Kubernetes for AI	14
Week 4	PyTorch for Accelerated Training	15
Week 5	TensorFlow Acceleration (gap-filler)	14
Week 6	LLM Fine-tuning & Optimization	15
Week 7	Distributed Training	15
Week 8	Scaling, Checkpointing & Deployment	15
Week 9	Accelerated Inference & Serving	16
Week 10	Distributed Data Engineering	16
Week 11	Smart-City Video Analytics	15
Week 12	Healthcare AI	15

Scoring tip: note any week under 70% and revise that week in the study guide before the exam.

Week 1 — AI Hardware & Accelerators

Assignment 1 concepts + roofline, CUDA grid and all-reduce calculations

1. A GPU has peak 1000 TFLOPS and memory bandwidth 3.35 TB/s. A kernel has arithmetic intensity 50 FLOP/byte. Using the roofline model, what is its attainable performance?

A) 1000 TFLOPS
B) 167.5 TFLOPS
C) 50 TFLOPS
D) 298 TFLOPS
Hint: Attainable = min(peak, intensity × bandwidth).

2. On the roofline plot, workloads to the LEFT of the ridge point are:

A) Compute-bound
B) Memory-bound
C) Latency-bound
D) I/O-bound

3. You train a model and nvidia-smi shows GPU utilisation around 40%. What is the most likely bottleneck per the Week 1 slides?

A) Tensor Cores are too slow
B) HBM bandwidth is saturated
C) The CPU / data pipeline cannot feed the GPU
D) NVLink is disabled

4. LLM inference (small batch) and DNN training are typically:

A) Compute-bound & memory-bound
B) Memory-bound & compute-bound
C) Both memory-bound
D) Both compute-bound

5. A vector of N = 1,000,000 elements is processed with 256 threads per block. How many blocks must be launched?

A) 3906
B) 3907
C) 4096
D) 256
Hint: blocks = (N + 255) / 256 with integer division.

6. With 4 GPUs and a 1 GB gradient, how much data does each GPU send in ring all-reduce?

A) 1 GB
B) 1.5 GB
C) 2 GB
D) 4 GB
Hint: $2(N-1)/N \times M$

7. ECC (SECDED) memory in AI servers:

A) Corrects single- and double-bit errors
B) Detects single-bit, corrects double-bit
C) Corrects single-bit, detects double-bit errors
D) Only detects errors

8. Which is NOT a CUDA Runtime API function?

A) cudaMalloc()
B) cudaMemcpy()
C) cuMemAlloc()
D) cudaFree()

9. A process on socket 0 reads memory attached to socket 1 in a NUMA server. Roughly what latency penalty does this add?

A) ~2×
B) ~4×
C) ~6×
D) No penalty

10. As Tensor Core precision drops from FP32 → FP16 → FP8, TFLOPS:

A) Decrease
B) Stay the same
C) Increase (precision is inversely proportional to TFLOPS)
D) Become unpredictable

11. What problem does NVSwitch primarily solve?

A) NVMe read limits
B) Bandwidth/scalability limits of point-to-point NVLink across many GPUs
C) PCIe power throttling
D) Ethernet congestion between racks

12. Your Docker image has no NVIDIA driver installed, yet `docker run --gpus all ... nvidia-smi` works. Why?

A) The image secretly contains the driver
B) The NVIDIA Container Toolkit injects /dev/nvidia* devices and host driver libraries at run time
C) Docker emulates the GPU
D) cuDNN replaces the driver

13. What does `torch.cuda.amp.autocast()` provide?

A) Forces FP64
B) Automatic Mixed Precision
C) Automatic model parallelism
D) Automatic checkpointing

Week 1 — Answer key

1. **B) 167.5 TFLOPS** — Ridge point = 1000/3.35 ≈ 298 FLOP/byte. 50 is left of the ridge → memory-bound. Attainable = 50 × 3.35 TB/s = 167.5 TFLOPS.
2. **B) Memory-bound** — Low arithmetic intensity (left) is limited by memory bandwidth; right of the ridge hits the compute ceiling. Note: the official Assignment 1 Q4 key words it as option B 'Compute intensive & memory bound' — pick that option if worded the same way.
3. **C) The CPU / data pipeline cannot feed the GPU** — Rule of thumb from the slides: if GPU utilisation is below ~60%, the CPU (data loading, preprocessing) is the bottleneck.
4. **B) Memory-bound & compute-bound** — Inference re-reads all weights for each token (low reuse) → memory-bound. Training with large batches has high data reuse → compute-bound.
5. **B) 3907** — 1,000,000 / 256 = 3906.25 → round up to 3907 so every element gets a thread (the extra threads are masked by `if (i < n)`).
6. **B) 1.5 GB** — 2 × (4−1)/4 × 1 GB = 1.5 GB. As N grows this approaches 2M.
7. **C) Corrects single-bit, detects double-bit errors** — Single Error Correction, Double Error Detection.
8. **C) cuMemAlloc()** — `cu*` functions belong to the lower-level Driver API; `cuda*` functions are the Runtime API.
9. **A) ~2×** — Cross-socket access traverses UPI/Infinity Fabric, adding about 2× latency. Pin affinity with numactl.
10. **C) Increase (precision is inversely proportional to TFLOPS)** — Lower precision packs more operations into the same silicon, so throughput rises.
11. **B) Bandwidth/scalability limits of point-to-point NVLink across many GPUs** — NVSwitch is an all-to-all crossbar so all 8 GPUs in a node communicate at full NVLink speed (900 GB/s per H100).
12. **B) The NVIDIA Container Toolkit injects /dev/nvidia* devices and host driver libraries at run time** — The host owns the driver; the toolkit's OCI hook mounts device files and libcuda into the container.
13. **B) Automatic Mixed Precision** — Runs matmuls in FP16/BF16 while keeping sensitive ops in FP32.

Week 2 — Containerized AI Systems

Docker, pinning, CUDA in containers, CI, Slurm/K8s scheduling, MIG/MPS

1. Every time you edit train.py, `docker build` reinstalls PyTorch and takes 30 minutes. What is the fix?

- **A)** Use a bigger base image
- **B)** COPY requirements.txt and pip install BEFORE copying the code
- **C)** Add --no-cache to the build
- **D)** Install the driver in the image

2. A container fails with 'CUDA driver version is insufficient for CUDA runtime version'. What's the cause?

- **A)** Host driver is older than the image's CUDA toolkit
- **B)** Image driver is too new
- **C)** cuDNN is missing
- **D)** Not enough /dev/shm

3. `nvidia-smi` works inside the container but `torch.cuda.is_available()` returns False. Most likely:

- **A)** Toolkit not installed
- **B)** PyTorch wheel built for a different CUDA (or CPU-only)
- **C)** GPU is broken
- **D)** Docker needs restarting

4. PyTorch DataLoader workers crash with 'bus error' inside a container. Fix?

- **A)** --gpus all
- **B)** --shm-size=8g
- **C)** -p 8888:8888
- **D)** --privileged

5. Which guarantees byte-level reproducibility of the base image?

- **A)** Tag `latest`
- **B)** Semantic version ranges
- **C)** Referencing the image by its SHA-256 digest
- **D)** A .lock suffix on the image name

6. In a multi-stage build, what happens to the builder stage's layers after `COPY --from=builder`?

- **A)** They become parent layers
- **B)** They are compressed into the last layer
- **C)** They are excluded from the final image
- **D)** They are stored as metadata

7. Slurm sets CUDA_VISIBLE_DEVICES=2,3 for your job. What do `torch.cuda.device_count()` and device 0 refer to?

- **A)** 8 devices; device 0 = physical GPU 0
- **B)** 2 devices; device 0 = physical GPU 2
- **C)** 2 devices; device 0 = physical GPU 0
- **D)** 4 devices; device 0 = physical GPU 2

8. Two teams share one H100 for inference. Team A's OOM must never affect Team B. Which mechanism?

A) Time-slicing
B) MPS
C) MIG
D) CUDA streams

9. A key limitation of CUDA MPS is:

A) Kernels can't run concurrently
B) Memory is shared, so a buggy process can corrupt others or crash the MPS server
C) Requires Blackwell GPUs
D) Maximum 2 clients

10. A 4-node job starts on 3 nodes while the 4th waits; the 3 nodes sit idle holding GPUs. What fixes this 'stranded GPU' problem?

A) Backfill
B) Gang scheduling (all-or-nothing)
C) Fair-share
D) Preemption

11. All-reduce within an NVLink domain runs at 900 GB/s; across PCIe at 32 GB/s. Wrong placement makes communication roughly how much slower?

A) ~3×
B) ~10×
C) ~28×
D) ~100×
Hint: 900 ÷ 32

12. In the 3-level GPU probe, what is the correct order?

A) torch.cuda.is_available → vector-add → nvidia-smi
B) nvidia-smi → vector-add compile/run → torch.cuda.is_available
C) vector-add → nvidia-smi → torch
D) Any order

13. In `kernel<<<X, Y>>>`, X and Y are:

A) Threads per block, blocks
B) Blocks in the grid, threads per block
C) Shared memory, constant memory
D) SMs, warps

14. Command to create a hash-locked requirements file with pip-tools?

A) pip freeze --hash
B) pip-compile --generate-hashes --output-file=requirements.txt requirements.in
C) pip-compile --lock-dependencies
D) pip install --require-hashes

Week 2 — Answer key

1. **B) COPY requirements.txt and pip install BEFORE copying the code** — Code edits invalidate their layer and all layers after it. Dependencies first, code last, so the install layer stays cached (30 min → 30 s).
2. **A) Host driver is older than the image's CUDA toolkit** — The host driver must be NEW enough for the toolkit. Fix: upgrade the host driver or use an older CUDA base image.
3. **B) PyTorch wheel built for a different CUDA (or CPU-only)** — Level 1 (driver) passes, level 3 (framework) fails → reinstall the wheel from the matching CUDA index.
4. **B) --shm-size=8g** — Workers exchange tensors via /dev/shm; Docker's 64 MB default is too small.
5. **C) Referencing the image by its SHA-256 digest** — Tags are mutable; digests are immutable.
6. **C) They are excluded from the final image** — Only the copied artifacts move over; compilers and caches (devel ~8 GB) stay out of the runtime image (~3 GB).
7. **B) 2 devices; device 0 = physical GPU 2** — Only listed GPUs are visible and they are re-indexed from 0.
8. **C) MIG** — MIG gives hard isolation: dedicated SMs, HBM and L2 per slice (up to 7). Time-slicing and MPS share memory.
9. **B) Memory is shared, so a buggy process can corrupt others or crash the MPS server** — MPS = soft isolation, up to 48 clients, Volta+.
10. **B) Gang scheduling (all-or-nothing)** — Slurm launches all tasks together; Kubernetes uses Volcano with minAvailable.
11. **C) ~28×** — 900/32 ≈ 28× — why topology-aware scheduling (`nvidia-smi topo -m`) matters.
12. **B) nvidia-smi → vector-add compile/run → torch.cuda.is_available** — Driver → toolkit → framework. Fix the first level that fails.
13. **B) Blocks in the grid, threads per block** — <<<gridDim, blockDim>>>.
14. **B) pip-compile --generate-hashes --output-file=requirements.txt requirements.in** — Intent in .in, truth in the generated .txt; install it with --require-hashes.

Week 3 — Kubernetes for AI

Control plane, GPU pods, services, ConfigMaps/Secrets, Gateway API, Istio canary

1. In the orchestration loop (declare → ? → reconcile), which phase continuously compares desired vs actual state per the official key?

A) You declare
B) It compiles
C) It reconciles
D) It watches

2. Which control-plane component is 'the only door in' (auth, validation, admission)?

A) etcd
B) API Server
C) Scheduler
D) kubelet

3. Your 3-worker distributed training StatefulSet hangs at NCCL initialisation. Workers are behind a normal ClusterIP Service. Fix?

A) Use a LoadBalancer Service
B) Use a headless Service (clusterIP: None) so each worker resolves to its own pod IP
C) Use NodePort
D) Increase replicas

4. A training pod uses `envFrom: secretRef: ml-tokens`, but you forgot to run create_secret.sh. What happens?

A) Pod starts without the variables
B) Empty values are generated
C) Pod stuck in CreateContainerConfigError
D) It falls back to a ConfigMap

5. A GPU pod sits in Pending forever on Docker Desktop's built-in Kubernetes. Why?

A) The image is too big
B) No NVIDIA device plugin, so no node advertises nvidia.com/gpu
C) restartPolicy is Never
D) Missing ConfigMap

6. GPU nodes are tainted `nvidia.com/gpu=present:NoSchedule`. What must a GPU pod include to land there?

A) A matching toleration
B) A ConfigMap
C) A headless Service
D) hostNetwork: true

7. Canary is set to 90/10. You send 40 requests. About how many should hit the canary?

A) ~4
B) ~10
C) ~20
D) ~36

8. Many training pods on different nodes must read AND write a shared dataset volume. Which access mode?

A) ReadWriteOnce
B) ReadOnlyMany
C) ReadWriteMany
D) emptyDir

9. Which manifest directive imports ALL key-values of a ConfigMap into the container env?

A) valueFrom
B) envFrom
C) dataFrom
D) configMapKeyRef

10. Which worker-node component programs iptables/eBPF so Services route to live pod IPs?

A) kubelet
B) containerd
C) kube-proxy
D) etcd

11. With Gateway API, who creates the HTTPRoute for /v1/completions?

A) The platform team, with cluster-admin
B) The ML/app team in its own namespace
C) The cloud provider
D) Istio automatically

12. team-alpha has a ResourceQuota of 2 GPUs. team-beta's GPUs are idle. Can team-alpha run 3 GPU pods?

A) Yes, quotas pool across namespaces
B) No, quotas are per namespace; the third pod is rejected
C) Yes, if it uses a toleration
D) Only at night

13. Kubernetes originated from which Google system?

A) Borg
B) Kubeflow
C) Helm
D) Argo

14. Which Service type is meant for dev/debug only and uses ports 30000–32767?

A) ClusterIP
B) NodePort
C) LoadBalancer
D) Headless

Week 3 — Answer key

1. **D) It watches** — Official Assignment 3 Q1 answer: 'It watches'.
2. **B) API Server** — Every kubectl call, webhook and controller goes through kube-apiserver; state is stored in etcd.
3. **B) Use a headless Service (clusterIP: None) so each worker resolves to its own pod IP** — ClusterIP gives one VIP for all workers; NCCL rendezvous needs stable per-pod names like worker-1.training-worker.
4. **C) Pod stuck in CreateContainerConfigError** — kubelet can't populate required env vars, so the container never starts.
5. **B) No NVIDIA device plugin, so no node advertises nvidia.com/gpu** — The device plugin (DaemonSet) advertises GPUs as extended resources; without it the scheduler finds no node. The lab used minikube with GPU support.
6. **A) A matching toleration** — Taints repel, tolerations allow, affinity attracts. Tainting keeps CPU pods off GPU nodes.
7. **A) ~4** — 40 × 10% = 4 (the lab observed 34 stable / 6 canary — random variation).
8. **C) ReadWriteMany** — RWX = many nodes read-write (NFS/EFS). RWO = one node. ROX = shared read-only weights.
9. **B) envFrom** — envFrom bulk-injects; valueFrom picks one key.
10. **C) kube-proxy** — kube-proxy handles Service networking.
11. **B) The ML/app team in its own namespace** — Platform owns GatewayClass + Gateway; app teams self-serve HTTPRoutes without cluster-admin.
12. **B) No, quotas are per namespace; the third pod is rejected** — ResourceQuota is a per-namespace ceiling enforced at admission.
13. **A) Borg** — Open-sourced 2014, donated to CNCF 2015.
14. **B) NodePort** — Production inference uses LoadBalancer.

Week 4 — PyTorch for Accelerated Training

Tensor shapes, autograd math, training-loop bugs, DataLoader, AMP

1. x = torch.randn(4, 1, 6); y = x.squeeze(); z = y.unsqueeze(1); w = z.permute(2, 0, 1). What is w.shape?

A) (4, 1, 6)
B) (6, 4, 1)
C) (1, 6, 4)
D) (6, 1, 4)
Hint: Track each step: squeeze removes size-1 dims.

2. x = torch.randn(2, 1, 3); y = x.unsqueeze(-1); z = y.squeeze(dim=0); w = z.squeeze(dim=-1). w.shape?

A) (2, 1, 3)
B) (1, 2, 3)
C) (2, 1, 3, 1)
D) (2, 3)

3. w = tensor(3.0, requires_grad=True), x = 2.0, L = w*x + w**2. After L.backward(), w.grad = ?

A) 6
B) 8
C) 9
D) 15

4. Which addition FAILS broadcasting?

A) (3,1) + (1,4)
B) (3,4) + (4,)
C) (3,4) + (3,)
D) (8,3,224,224) + (1,3,1,1)

5. How many parameters does nn.Linear(784, 512) have?

A) 401,408
B) 401,920
C) 784,512
D) 512

6. In `__init__`, a student writes `self.relu = nn.ReLU`. What happens when the model runs?

A) Works fine
B) Fails — the class isn't instantiated (needs nn.ReLU())
C) ReLU is skipped silently
D) Uses GELU instead

7. Loss oscillates wildly and explodes from the first few steps; the code never calls optimizer.zero_grad(). Why?

A) Gradients accumulate across steps
B) Learning rate resets
C) Dropout is disabled
D) Weights are frozen

8. A multi-class classifier outputs raw logits. Which loss and label format?

A) BCEWithLogitsLoss, float labels
B) CrossEntropyLoss, integer class labels (no softmax layer)
C) MSELoss, one-hot
D) CrossEntropyLoss after a Softmax layer

9. `nvidia-smi dmon` shows SM% oscillating 0% → 95% → 0%. What should you try first?

A) Larger model
B) Increase num_workers and set pin_memory=True
C) Disable AMP
D) Lower learning rate

10. Your benchmark reports 0.1 ms per training step on a big model. What's wrong?

A) torch.compile is on
B) Missing torch.cuda.synchronize() — you timed async CPU dispatch
C) TF32 is enabled
D) Nothing, it's correct

11. Training with BF16 autocast on an A100. Do you need GradScaler?

A) Yes, always
B) No — BF16 has FP32's exponent range
C) Only for CNNs
D) Only during evaluation

12. Which DataLoader argument page-locks host memory for faster DMA to the GPU?

A) persistent_workers
B) drop_last
C) pin_memory
D) prefetch_factor

13. After adding torch.compile, the first 3–5 steps are SLOWER than eager. Why?

A) Compilation happens on the first forward passes
B) Bug in the model
C) GPU warm-up
D) Dropout

14. What does model.eval() change?

A) Clears gradients
B) Enables AMP
C) Disables Dropout and makes BatchNorm use running statistics
D) Stops gradient computation

15. Which LR scheduler smoothly decays the learning rate along a cosine curve?

A) StepLR
B) CosineAnnealingLR
C) Static scheduling
D) Ignore-index scheduling

Week 4 — Answer key

1. B) (6, 4, 1) — squeeze → (4,6); unsqueeze(1) → (4,1,6); permute(2,0,1) → (6,4,1).

2. A) (2, 1, 3) — squeeze(0) does nothing because dim 0 has size 2. (Official Assignment 4 Q1.)

3. B) 8 — dL/dw = x + 2w = 2 + 6 = 8.

4. C) (3,4) + (3,) — Align from the right: 4 vs 3 mismatch. Use b.unsqueeze(-1) → (3,1).

5. B) 401,920 — 784 × 512 weights + 512 biases = 401,408 + 512 = 401,920.

6. B) Fails — the class isn't instantiated (needs nn.ReLU()) — Layers must be instances. (Official Assignment 4 Q5.)

7. A) Gradients accumulate across steps — PyTorch adds gradients into .grad each backward(); step N uses steps 1..N combined.

8. B) CrossEntropyLoss, integer class labels (no softmax layer) — CrossEntropyLoss applies log-softmax internally.

9. B) Increase num_workers and set pin_memory=True — GPU is starving for data — a DataLoader bottleneck. The dummy-dataset trick confirms it.

10. B) Missing torch.cuda.synchronize() — you timed async CPU dispatch — CUDA kernels are asynchronous; synchronize before starting and stopping the timer.

11. B) No — BF16 has FP32's exponent range — Use FP16 + GradScaler on older GPUs like V100.

12. C) pin_memory — pin_memory=True.

13. A) Compilation happens on the first forward passes — Don't benchmark early steps; later steps run fused compiled code (20–50% faster).

14. C) Disables Dropout and makes BatchNorm use running statistics — Use torch.no_grad() separately to stop gradient tracking.

15. B) CosineAnnealingLR — OneCycleLR = warm-up then cosine decay.

Week 5 — TensorFlow Acceleration (gap-filler)

Not from your repo: built from the course outline (tf.distribute, XLA, mixed precision, tf.data)

1. You have one machine with 4 GPUs and want synchronous data-parallel training in TensorFlow. Which strategy?

- **A)** tf.distribute.MirroredStrategy
- **B)** tf.distribute.TPUStrategy
- **C)** tf.distribute.ParameterServerStrategy
- **D)** tf.distribute.OneDeviceStrategy

2. For synchronous training across multiple machines, which strategy and how are workers configured?

- **A)** MirroredStrategy + MASTER_ADDR
- **B)** MultiWorkerMirroredStrategy + TF_CONFIG environment variable
- **C)** TPUStrategy + torchrun
- **D)** OneDeviceStrategy + Slurm

3. Per-replica batch size is 64 and strategy.num_replicas_in_sync = 4. What is the global batch size?

- **A)** 16
- **B)** 64
- **C)** 256
- **D)** 68

4. Base LR 0.001 was tuned at batch 64 on one GPU. You move to 8 GPUs with per-replica batch 64. Using the linear scaling rule, new LR?

- **A)** 0.001
- **B)** 0.004
- **C)** 0.008
- **D)** 0.064

5. Where must the Keras model and optimizer be created when using a tf.distribute strategy?

- **A)** Anywhere
- **B)** Inside `with strategy.scope():`
- **C)** Inside tf.function
- **D)** After model.fit()

6. What does decorating a Python function with @tf.function do?

- **A)** Runs it eagerly line by line
- **B)** Traces it into a TensorFlow graph for faster, optimisable execution
- **C)** Distributes it across GPUs
- **D)** Quantizes it

7. Which option enables XLA compilation (op fusion) for a Keras model or tf.function?

- **A)** jit_compile=True
- **B)** run_eagerly=True
- **C)** allow_growth=True
- **D)** use_xla=False

8. How do you enable mixed precision in Keras on a modern NVIDIA GPU, and what should the output layer use?

A) set_global_policy('float64'); output float16
B) set_global_policy('mixed_float16'); keep the final output/softmax in float32
C) model.half()
D) No change needed

9. GPU utilisation is low because preprocessing runs between steps. Which tf.data call overlaps input preparation with training?

A) .prefetch(tf.data.AUTOTUNE)
B) .repeat()
C) .shuffle(1)
D) .take(1)

10. TensorFlow grabs all GPU memory at start, blocking a second process. What setting avoids this?

A) tf.config.experimental.set_memory_growth(gpu, True)
B) jit_compile=True
C) tf.data.AUTOTUNE
D) MirroredStrategy

11. Which strategy trains asynchronously with workers and parameter servers?

A) MirroredStrategy
B) MultiWorkerMirroredStrategy
C) ParameterServerStrategy
D) TPUStrategy

12. You need to deploy a TF model to (i) a mobile phone and (ii) maximum-throughput NVIDIA GPU inference. Which pair?

A) (i) TFLite, (ii) TF-TRT / TensorRT
B) (i) TF-TRT, (ii) TFLite
C) (i) SavedModel only, (ii) Horovod
D) (i) ONNX, (ii) TPUStrategy

13. PyTorch vs TensorFlow: which pairing is correct?

A) DDP ↔ MirroredStrategy; torch.compile ↔ tf.function/XLA; autocast ↔ mixed_float16 policy
B) DDP ↔ TFLite; torch.compile ↔ tf.data
C) FSDP ↔ OneDeviceStrategy
D) DataLoader ↔ XLA

14. A Horovod script that works with TensorFlow needs which launcher?

A) torchrun
B) horovodrun (wraps mpirun)
C) kubectl apply
D) srun only

Week 5 — Answer key

1. **A) tf.distribute.MirroredStrategy** — MirroredStrategy replicates the model on each local GPU and all-reduces gradients (NCCL by default) — TF's equivalent of single-node DDP.
2. **B) MultiWorkerMirroredStrategy + TF_CONFIG environment variable** — TF_CONFIG holds the cluster spec (worker addresses) and this worker's task type/index.
3. **C) 256** — Global batch = per-replica batch × replicas = 64 × 4 = 256 (same idea as DDP's effective batch).
4. **C) 0.008** — Global batch 512 = 8× → LR × 8 = 0.008 (add warm-up).
5. **B) Inside** `with strategy.scope():` — Variables created under the scope are mirrored across replicas.
6. **B) Traces it into a TensorFlow graph for faster, optimisable execution** — Graph mode removes Python overhead — analogous to torch.compile / graph capture.
7. **A) jit_compile=True** — XLA (Accelerated Linear Algebra) fuses ops into fewer kernels, reducing memory traffic.
8. **B) set_global_policy('mixed_float16'); keep the final output/softmax in float32** — Compute in FP16 on Tensor Cores; keep numerically sensitive outputs in FP32. Keras wraps the optimizer with loss scaling ('mixed_bfloat16' on TPU/Ampere+ needs none).
9. **A) .prefetch(tf.data.AUTOTUNE)** — prefetch prepares the next batch while the current one trains; also use map(num_parallel_calls=AUTOTUNE), cache(), interleave().
10. **A) tf.config.experimental.set_memory_growth(gpu, True)** — Memory growth allocates as needed instead of reserving the whole GPU.
11. **C) ParameterServerStrategy** — Mirrored variants are synchronous all-reduce; parameter-server is async.
12. **A) (i) TFLite, (ii) TF-TRT / TensorRT** — SavedModel is the standard export; TFLite for mobile/edge; TF-TRT integrates TensorRT (it optimizes the largest supported subgraphs — 2022 Week 9 Q9).
13. **A) DDP ↔ MirroredStrategy; torch.compile ↔ tf.function/XLA; autocast ↔ mixed_float16 policy** — Also DataLoader ↔ tf.data, torchrun env vars ↔ TF_CONFIG, Horovod works with both.
14. **B) horovodrun (wraps mpirun)** — Horovod uses MPI for rendezvous and NCCL for collectives; framework-agnostic (TF, PyTorch, MXNet). Covered in Week 7.

Week 6 — LLM Fine-tuning & Optimization

Mixed precision, memory math, batching waste, ZeRO, RAG vs fine-tune, LoRA/QLoRA

1. How much memory do the weights alone of a 7B model take in BF16?

A) 7 GB
B) 14 GB
C) 28 GB
D) 56 GB

2. Microbatch 4 per GPU, gradient accumulation 8 steps, 2 GPUs. Effective batch?

A) 14
B) 32
C) 64
D) 128

3. A microbatch has sequence lengths [1024, 256, 256, 512], padded to 1024. What fraction of token positions is padding?

A) 25%
B) 50%
C) 57%
D) 75%

4. LoRA with rank r = 8 on a 4096 × 4096 weight. How many trainable parameters for that matrix?

A) 32,768
B) 65,536
C) 16.7 million
D) 4,096
Hint: A is r × d, B is d × r.

5. Main benefit of BF16 over FP16 for training transformers?

A) Fewer bits
B) Wider exponent range
C) More mantissa precision
D) Automatic loss scaling

6. Logits [1000, 1001, 999] give NaN in softmax. Fix?

A) model.half()
B) Subtract the max logit before softmax
C) Run backward in autocast
D) Use a legacy GPU

7. FP16 training: some layers stop learning because tiny gradients become exactly 0. This is:

A) Overflow
B) Underflow
C) Cancellation
D) Drift

8. Dynamic loss scaler sees inf in gradients. What happens?

A) Step is skipped and the scale reduced
B) Only bad weights skipped
C) Falls back to FP32
D) Training stops

9. OOM happens only during backward on long contexts. First lever to try?

A) 8-bit optimizer
B) Activation checkpointing or smaller microbatch
C) More gradient accumulation only
D) Bigger learning rate

10. Which ZeRO stage shards optimizer states AND gradients, but not parameters?

A) ZeRO-1
B) ZeRO-2
C) ZeRO-3
D) DDP

11. An enterprise assistant must answer from internal documents that change every week. Best first lever?

A) Full fine-tuning weekly
B) RAG
C) Continued pretraining
D) QLoRA

12. How are LoRA matrices A and B initialised?

A) A zeros, B Gaussian
B) A Gaussian, B zeros
C) Both zeros
D) Both uniform

13. Which are QLoRA's three innovations?

A) FP8, ZeRO-3, SDPA
B) 4-bit NormalFloat, double quantization, paged optimizers
C) GPTQ, AWQ, GGUF
D) Pruning, distillation, LoRA

14. Which component has exactly the same shape as the model weights?

A) Activations
B) Gradients
C) KV cache
D) Token IDs

15. Base GPT-2 given a zero-shot instruction mostly continues the prompt. Why?

A) It can't write English
B) It was never instruction-tuned — it only learned next-token continuation
C) Safety policy
D) Tokenizer bug

Week 6 — Answer key

1. **B) 14 GB** — 7B × 2 bytes = 14 GB (FP32 = 28 GB). Training adds gradients, optimizer states and activations.
2. **C) 64** — 4 × 8 × 2 = 64.
3. **B) 50%** — 4 × 1024 = 4096 positions; useful = 2048 → 50% waste. Fix with bucketing or sequence packing.
4. **B) 65,536** — r × (d + d) = 8 × 8192 = 65,536 ≈ 0.39% of 16.7M.
5. **B) Wider exponent range** — BF16 = 8 exponent / 7 mantissa (FP32-like range); FP16 = 5/10.
6. **B) Subtract the max logit before softmax** — [−1, 0, −2] gives identical probabilities without overflow; also prefer log_softmax/logsumexp.
7. **B) Underflow** — Fix with GradScaler (loss scaling) or BF16.
8. **A) Step is skipped and the scale reduced** — Official Assignment 6 Q10.
9. **B) Activation checkpointing or smaller microbatch** — Backward OOM = saved activations. Checkpointing trades ~20% compute for memory. OOM after first optimizer step → optimizer states.
10. **B) ZeRO-2** — ZeRO-3 / FSDP FULL_SHARD adds parameters.
11. **B) RAG** — Stale knowledge → RAG (no weight change). Output format → SFT. Tone/ranking → preference tuning.
12. **B) A Gaussian, B zeros** — B = 0 makes ΔW = BA = 0 at start, so the model begins identical to the pretrained one.
13. **B) 4-bit NormalFloat, double quantization, paged optimizers** — 65B fine-tuning drops from 780 GB to 48 GB on one GPU.
14. **B) Gradients** — One gradient per parameter.
15. **B) It was never instruction-tuned — it only learned next-token continuation** — Official Assignment 6 Q12.

Week 7 — Distributed Training

Parallelism choice, DDP gotchas, ring all-reduce and scaling-efficiency calculations

1. 8 GPUs, 4 GB gradient. Per-GPU traffic in ring all-reduce?

A) 4 GB
B) 7 GB
C) 8 GB
D) 32 GB

2. 1 GPU: 500 samples/s. 4 GPUs: 1800 samples/s. Scaling efficiency?

A) 72%
B) 90%
C) 360%
D) 111%

3. 3 nodes × 8 GPUs. What is WORLD_SIZE, and the global RANK of the process with node_rank=1, local_rank=2?

A) 24 and 10
B) 8 and 2
C) 24 and 3
D) 3 and 10

4. 3D parallelism with TP = 8, PP = 4, DP = 4 uses how many GPUs?

A) 16
B) 32
C) 128
D) 256

5. Your DDP model's validation accuracy is mysteriously worse than single-GPU, and every epoch sees batches in the same order. Cause?

A) Forgot sampler.set_epoch(epoch)
B) Used NCCL
C) drop_last=True
D) Saved from rank 0

6. Your DDP job hangs at the end of each epoch: one rank finished early, others wait in all-reduce. Fix?

A) find_unused_parameters=True
B) drop_last=True in DistributedSampler
C) Bigger bucket size
D) Use Gloo

7. A saved DDP checkpoint fails to load into a plain model with 'module.' key errors. Fix?

A) Save model.module.state_dict()
B) Save optimizer too
C) Use safetensors
D) Set strict=True

8. Multi-node torchrun times out at rendezvous. Most likely cause?

A) Learning rate too high
B) Firewall blocking MASTER_PORT, wrong MASTER_ADDR, or world-size mismatch
C) Too many DataLoader workers
D) BF16 enabled

9. Model is 200B parameters; the largest single layer fits on one GPU. Which approach?

A) DDP
B) Tensor parallelism
C) FSDP / ZeRO-3
D) Pipeline only

10. You suspect InfiniBand is misconfigured. Which env var forces NCCL to TCP so you can compare?

A) NCCL_DEBUG=INFO
B) NCCL_IB_DISABLE=1
C) NCCL_SOCKET_IFNAME=eth0
D) NCCL_BUFFSIZE=0

11. You keep 8 GB per GPU of work fixed and grow the cluster to train a bigger model in the same wall time. This measures:

A) Strong scaling
B) Weak scaling
C) Amdahl scaling
D) Elastic scaling

12. Per the slides, which lever dominates communication efficiency?

A) Hierarchical all-reduce
B) Larger per-GPU batch + gradient accumulation
C) Gradient compression
D) AMP

13. Order these interconnects from fastest to slowest.

A) NVLink > InfiniBand > RoCEv2 > PCIe > TCP
B) TCP > PCIe > NVLink
C) InfiniBand > NVLink > TCP
D) PCIe > NVLink > InfiniBand

14. DDP per-rank memory for 7B with AdamW: which component is largest?

A) BF16 weights (14 GB)
B) BF16 gradients (14 GB)
C) FP32 optimizer m + v (56 GB)
D) NCCL buffers

15. How does DDP isolate work across GPUs?

A) One thread per GPU
B) One process per GPU
C) One process for all GPUs
D) Many processes per GPU

Week 7 — Answer key

1. **B) 7 GB** — 2 × 7/8 × 4 GB = 7 GB. Naive all-reduce would be O(N×M).
2. **B) 90%** — 1800 / (4 × 500) = 0.90.
3. **A) 24 and 10** — World = 3 × 8 = 24. Rank = node_rank × gpus_per_node + local_rank = 8 + 2 = 10.
4. **C) 128** — Dimensions multiply: 8 × 4 × 4 = 128.
5. **A) Forgot sampler.set_epoch(epoch)** — DistributedSampler reuses the same shuffle seed without set_epoch.
6. **B) drop_last=True in DistributedSampler** — Uneven final batch → hang.
7. **A) Save model.module.state_dict()** — DDP wraps the model and prefixes keys with 'module.'.
8. **B) Firewall blocking MASTER_PORT, wrong MASTER_ADDR, or world-size mismatch** — Test with `telnet node0 29500` from each node. Rank 0 opens a TCP store others must reach.
9. **C) FSDP / ZeRO-3** — Memory is the constraint, not any one layer. One layer too big → tensor parallel.
10. **B) NCCL_IB_DISABLE=1** — NCCL_DEBUG=INFO shows which transport (NVL, SHM, IB, socket) was picked.
11. **B) Weak scaling** — Strong = fixed problem, finish faster.
12. **B) Larger per-GPU batch + gradient accumulation** — Amortises each all-reduce over more compute. (AMP is #1 by ROI overall.)
13. **A) NVLink > InfiniBand > RoCEv2 > PCIe > TCP** — TCP sockets are the slowest fallback (10–25 Gb/s).
14. **C) FP32 optimizer m + v (56 GB)** — 7B × 4 bytes × 2 = 56 GB, duplicated on every rank — what FSDP shards away.
15. **B) One process per GPU** — Avoids the Python GIL.

Week 8 — Scaling, Checkpointing & Deployment

LR-scaling math, failure odds, checkpoints, packaging, rollouts, SLOs + 2022 items

1. LR 0.075 at batch 512. Batch increases to 4096. Linear-scaling LR?

A) 0.2
B) 0.6
C) 0.75
D) 0.8

2. Tuned on 1 GPU: batch 32, LR 1e-4. Now 16 GPUs, per-GPU batch 32, gradient accumulation 2. New LR (linear rule)?

A) 1.6e-3
B) 3.2e-3
C) 6.4e-3
D) 1e-4

3. Using Adam with the square-root rule: LR 1e-4 at batch 256 → batch 4096. New LR?

A) 1.6e-3
B) 4e-4
C) 2e-4
D) 1e-4

4. Checkpoints every 500 steps. A GPU fails at step 1720. torchrun restarts. Where does training resume and how much work is lost?

A) Step 1720, none lost
B) Step 1500, 220 steps lost
C) Step 0
D) Step 2000

5. Each node fails with probability 0.1% per day. With 256 nodes, chance of at least one failure today?

A) ~0.1%
B) ~2.6%
C) ~22%
D) ~97%

6. After resuming, the run converges worse than the uninterrupted one. What was most likely NOT saved?

A) Model weights
B) Optimizer, scheduler, RNG / sampler or AMP scaler state
C) The model card
D) The Dockerfile

7. A synchronous save stalls GPUs ~60 s. What reduces it to ~2 s?

A) Full checkpoint to rank 0
B) Async checkpointing (async_save)
C) Saving less often
D) Pickle

8. A security scan flags your shared model file because loading it can execute code. Switch to:

A) pickle
B) safetensors
C) TorchScript
D) .zip

9. You must duplicate live requests to a new model and compare outputs WITHOUT returning its answers to users. Strategy?

A) Canary
B) Shadow
C) Blue/green
D) Rolling

10. SLO: 99.9% of requests under 500 ms over 30 days. Error budget?

A) ~4.3 min
B) ~43 min
C) ~7.2 h
D) ~1 day

11. Pods crash-loop right after deploy because the model takes 3 minutes to load. Fix?

A) Add a startupProbe with a generous failureThreshold
B) Remove readinessProbe
C) Use the latest tag
D) maxUnavailable: 3

12. Loss jumps 50× right as warm-up ends on an FP16 run. Reasonable recovery?

A) Restart from scratch
B) Clip gradients (1.0), switch to BF16, roll back to last checkpoint and skip the batch
C) Increase LR
D) Remove warm-up

13. Why are optimizer states stripped before shipping an artifact?

A) They increase size 4–5× and are useless for inference
B) They break ONNX
C) Legal reasons
D) They slow training

14. (2022) Which TensorRT feature maximises throughput with FP16/INT8?

A) Time fusion
B) Reduced precision / precision calibration
C) Auto-tuning kernel
D) Dynamic tensor memory

15. (2022) Transferring the generalisation of a big model into a compact model is:

A) Pruning
B) Knowledge distillation
C) Quantization
D) Tuning

Week 8 — Answer key

1. **B) 0.6** — 4096/512 = 8 → 0.075 × 8 = 0.6. (Official Assignment 8 Q1.)
2. **B) 3.2e-3** — Effective batch = 32 × 16 × 2 = 1024 = 32× → 1e-4 × 32 = 3.2e-3, with warm-up.
3. **B) 4e-4** — √(4096/256) = √16 = 4 → 4e-4.
4. **B) Step 1500, 220 steps lost** — Latest checkpoint = 1500; counter resumes at 1501.
5. **C) ~22%** — 1 − 0.999^256 ≈ 22%. Over 14 days ≈ 97%.
6. **B) Optimizer, scheduler, RNG / sampler or AMP scaler state** — A correct checkpoint holds weights + optimizer + scheduler + step/epoch + RNG + sampler + scaler.
7. **B) Async checkpointing (async_save)** — Copy to pinned memory, write in background. Sharded DCP checkpoints also write in parallel.
8. **B) safetensors** — Pure numeric, zero-copy, no code execution.
9. **B) Shadow** — Canary serves 1–5% of real traffic; blue/green flips stacks; rolling is the K8s default.
10. **B) ~43 min** — 0.1% × 30 × 24 × 60 = 43.2 min.
11. **A) Add a startupProbe with a generous failureThreshold** — Slides: failureThreshold 60 × 5 s = 5 min grace.
12. **B) Clip gradients (1.0), switch to BF16, roll back to last checkpoint and skip the batch** — Spike-and-resume is routine at scale (e.g. OPT-175B).
13. **A) They increase size 4–5× and are useless for inference** — Checkpoint = resume; artifact = weights only.
14. **B) Reduced precision / precision calibration** — 2022 Week 8 accepted answer.
15. **B) Knowledge distillation** — Pruning removes weights; 'Tuning' is NOT a compression technique.

Week 9 — Accelerated Inference & Serving

KV cache and decode math, serving engines, graph capture, ONNX/TensorRT, quantization

1. KV cache for one request: 32 layers, 32 heads, d_head 128, sequence 4096, FP16 (2 bytes). Size?

A) ~0.5 GB
B) ~1 GB
C) ~2 GB
D) ~16 GB
Hint: 2 × layers × heads × d_head × seq × bytes

2. A 7B BF16 model (14 GB) decodes at batch 1 on an H100 (3.35 TB/s). Theoretical max tokens/s?

A) ~24
B) ~240
C) ~2400
D) ~3958

3. Decode at batch 1 has arithmetic intensity ~1 FLOP/byte vs H100 ridge ~1181. The main lever to raise throughput is:

A) Faster Tensor Cores
B) Batching more requests
C) Longer prompts
D) FP64

4. Which metric captures prefill latency that users feel as 'responsiveness'?

A) TPOT
B) TTFT
C) MFU
D) QPS

5. Traditional serving pre-allocates KV for 4096 tokens but outputs average 200. Which technique fixes the waste?

A) PagedAttention (16-token pages allocated on demand)
B) Static batching
C) torch.jit.trace
D) Group-wise quantization

6. A RAG agent sends thousands of requests sharing the same long system prompt. Which framework feature helps most?

A) SGLang RadixAttention (prefix KV reuse)
B) CUDA streams
C) TF32
D) Gradient checkpointing

7. A small draft model proposes K tokens and the big model verifies them in one pass. This is:

A) Continuous batching
B) Speculative decoding
C) Chunked prefill
D) Flash decoding

8. At batch 1, step time is 25 ms: 20 ms CPU kernel launches + 5 ms GPU. What fixes this?

A) CUDA Graphs (e.g. torch.compile mode='reduce-overhead')
B) Bigger HBM
C) jit.trace only
D) More DataLoader workers

9. A model has `if x.sum() > 0:` in forward and needs variable batch sizes. Which capture API?

A) torch.jit.trace
B) torch.export with dynamic shapes
C) CUDA graph
D) pickle

10. ONNX Runtime with ['CUDAExecutionProvider', 'CPUExecutionProvider'] runs slowly; one op isn't supported on CUDA. Why slow?

A) Fallback to CPU adds data transfers and synchronization
B) Fusions are disabled forever
C) Model parameters shrink
D) Opset is too new

11. In TensorRT, 'tactic selection' means:

A) Fusing layers
B) Benchmarking candidate kernels and choosing the fastest
C) Choosing min/opt/max shapes
D) Quantizing weights

12. Raw weight memory of a 13B model at INT4?

A) ~3.25 GB
B) ~6.5 GB
C) ~13 GB
D) ~26 GB

13. Which method uses approximate second-order (Hessian) information to compensate layer-wise quantization error?

A) AWQ
B) GPTQ
C) Dynamic PTQ
D) NF4

14. In asymmetric quantization, the 'zero point' is:

A) Max integer value
B) The integer code that represents real 0.0
C) Spacing between levels
D) Reconstruction error

15. PTQ loses too much accuracy for your model. Next step?

A) QAT — simulate quantization during training/fine-tuning
B) Bigger group size
C) Per-tensor scale
D) Ship anyway

16. (2022) Columnar in-memory format for fast data interchange?

A) Apache Hive
B) Apache Arrow
C) Apache Spark
D) Apache HBase

Week 9 — Answer key

1. **C) ~2 GB** — 2 × 32 × 32 × 128 × 4096 × 2 = 2,147,483,648 bytes ≈ 2 GiB.
2. **B) ~240** — Every token reads all weights: 3350 / 14 ≈ 239 tokens/s (70B → ~24).
3. **B) Batching more requests** — Batch 64 raises intensity ~64×; continuous batching gives 10–23×.
4. **B) TTFT** — TTFT = time to first token (prefill, compute-bound). TPOT = per-token decode (memory-bound).
5. **A) PagedAttention (16-token pages allocated on demand)** — vLLM's PagedAttention → 3–4× more concurrent requests. TensorRT-LLM calls it paged KV cache.
6. **A) SGLang RadixAttention (prefix KV reuse)** — Up to 5× faster on prefix-heavy workloads.
7. **B) Speculative decoding** — ~80% acceptance → 2–3× speed-up.
8. **A) CUDA Graphs (e.g. torch.compile mode='reduce-overhead')** — Replay all kernels with one launch → ~5.3 ms (~5×). Requires static shapes, no .item().
9. **B) torch.export with dynamic shapes** — trace records one path and is shape-specific. jit.script handles control flow; torch.export is the 2.x recommendation.
10. **A) Fallback to CPU adds data transfers and synchronization** — ORT is great for cross-platform portability, less so for max NVIDIA throughput.
11. **B) Benchmarking candidate kernels and choosing the fastest** — Why engine builds are expensive. Dynamic shapes need optimization profiles (MIN/OPT/MAX).
12. **B) ~6.5 GB** — 13B × 0.5 byte = 6.5 GB floor; real memory adds scales, KV cache and buffers.
13. **B) GPTQ** — AWQ instead uses calibration data to protect salient activation channels.
14. **B) The integer code that represents real 0.0** — Ensures zero padding is exact. Group-wise scales minimise error.
15. **A) QAT — simulate quantization during training/fine-tuning** — Try PTQ first, QAT when accuracy is unacceptable. LLMs are hard because of outlier channels.
16. **B) Apache Arrow** — 2022 set also: RAPIDS, cuML (sklearn on GPU), cuDF, cuGraph, Thrust, cuXfilter, UFF.

Week 10 — Distributed Data Engineering

Partitioning, shuffle, skew/salting, Dask vs Ray, Spark, cuDF, observability

1. Clickstream data will be grouped by user_id. Which partitioning gives zero shuffle for that group-by?

A) Row range
B) Hash(user_id)
C) Range(date)
D) Random

2. A Spark stage shows one task at 30 min while 199 tasks finish in 30 s. Diagnosis and fix?

A) Network loss; add retries
B) Data skew; salted group-by or enable AQE skew join
C) Too many partitions; coalesce
D) Driver OOM; add memory

3. In the salted group-by pattern, why are there two aggregations?

A) First by (key, salt) to spread the hot key; second by key to combine the partial sums
B) To delete duplicates
C) To convert UDFs to SQL
D) To cache data

4. A stage reads 2,000,000 rows and outputs 1,500,000. drop_rate? Should it alert at the slide's 0.1 threshold?

A) 0.25 — yes, alert
B) 0.75 — no
C) 0.25 — no
D) 0.5 — yes

5. Pandas groupby on 80M rows: 25 s on 32 cores. cuDF on A100: 0.4 s. Speed-up?

A) ~6×
B) ~62×
C) ~10×
D) ~625×

6. You need GPU batch inference over 1B images with the model loaded once per GPU. Best tool?

A) Dask DataFrame
B) Ray Data with GPU actors
C) Pandas
D) Spark SQL

7. Tabular ETL of 2 TB written in pandas style. Easiest scale-out?

A) Dask DataFrame (pd → dd)
B) Ray Tune
C) cuDF on one GPU
D) Horovod

8. Why is Dask/Spark faster than pandas for chained filter + groupby on big data?

A) Eager execution
B) Lazy evaluation builds a DAG optimised before running (pushdowns)
C) Static allocation
D) Row-range lookups

9. A worker dies mid-groupBy. How does the scheduler notice, and how is data recovered?

A) Heartbeats stop; lost partitions recomputed from lineage
B) Primary-key scan; restart job
C) User report; manual checkpoint
D) It can't recover

10. Spark driver crashes with OOM after a notebook cell. Most likely mistake?

A) Enabling AQE
B) Calling .collect() on a huge DataFrame
C) Writing to Parquet
D) Using broadcast joins

11. Joining a 2 TB table with a 50 MB lookup table. How to avoid shuffling both sides?

A) Broadcast the small table
B) Global sort first
C) Salt both
D) Use .collect()

12. Which Spark component automatically rewrites queries (filter pushdown, column pruning)?

A) Executor
B) Catalyst Optimizer
C) Delta Lake
D) Cluster manager

13. 'ERROR partition 47 OOM at 02:13' is an example of a:

A) Metric
B) Log
C) Trace
D) SLO

14. Nightly ETL jumped from 40 min to 3 h. Official root cause in the course scenario?

A) Disk failure
B) Data skew from a new bot account
C) Packet loss
D) Wrong schema

15. Per the official Assignment 10 key, in what order should the three signals be analysed?

A) Traces, Metrics, Logs
B) Metrics, Traces, Logs
C) Logs, Metrics, Traces
D) Traces, Logs, Metrics

16. (2022) Uniform API over InfiniBand, TCP/shared memory and NVLink?

A) OpenUCX
B) OpenMPI
C) OpenVINO
D) OpenAPI

Week 10 — Answer key

1. **B) Hash(user_id)** — All rows of a user land in the same partition. Sensor data queried by date → Range(date).
2. **B) Data skew; salted group-by or enable AQE skew join** — One hot key overloads one partition.
3. **A) First by (key, salt) to spread the hot key; second by key to combine the partial sums** — Two shuffles instead of one, but balanced; the second is small.
4. **A) 0.25 — yes, alert** — (2.0M − 1.5M)/2.0M = 0.25 > 0.1 → unexpected drop.
5. **B) ~62×** — 25/0.4 = 62.5 (slides round to ~50×). Below ~100k rows cuDF gives no benefit.
6. **B) Ray Data with GPU actors** — Ray actor = stateful object keeping the model resident. Ray tasks reload per call. Dask for GPU inference is a listed mistake.
7. **A) Dask DataFrame (pd → dd)** — Dask fits 10 GB–10 TB ETL; deploy on Kubernetes for autoscaling pods.
8. **B) Lazy evaluation builds a DAG optimised before running (pushdowns)** — Calling .compute() too often loses this benefit.
9. **A) Heartbeats stop; lost partitions recomputed from lineage** — Persist/cache key stages for long DAGs.
10. **B) Calling .collect() on a huge DataFrame** — Write to storage instead of pulling everything to the driver.
11. **A) Broadcast the small table** — Slides: broadcast tables under ~100 MB.
12. **B) Catalyst Optimizer** — Spark hierarchy: job → stages (no shuffle inside) → tasks (one partition each).
13. **B) Log** — Log = discrete timestamped event; metric = number over time; trace = where time went.
14. **B) Data skew from a new bot account** — One executor's partition had 40× the median rows.
15. **C) Logs, Metrics, Traces** — Official key. (Slide example went metrics → logs → traces; prefer the key if the wording matches.)
16. **A) OpenUCX** — 2022 set also: NVTabular, Spark 3.0 (ETL + DL), Spark Shuffle, EGX cluster = 8 nodes.

Week 11 — Smart-City Video Analytics

Detection vs tracking, event rules with geometry/time math, evaluation, plus 2022 DeepStream items

1. A video is 1920×1080. A ROI vertex is stored as normalised [0.55, 0.25]. Pixel coordinates?

A) (1056, 270)
B) (594, 480)
C) (1056, 480)
D) (270, 1056)

2. Track 14 enters the restricted zone at t = 12.4 s; loiter_seconds = 10. Earliest loitering event (if it stays)?

A) 10.0 s
B) 12.4 s
C) 22.4 s
D) 24.8 s

3. Per frame: decode 5 ms, inference 15 ms, tracking 3 ms, analytics + drawing 4 ms, encoding 6 ms (sequential). End-to-end FPS?

A) ~66 FPS
B) ~30 FPS
C) ~200 FPS
D) ~15 FPS

4. A labelled clip has 8 true events. The system emits 10, of which 6 are correct. Event precision and recall?

A) P 0.75, R 0.60
B) P 0.60, R 0.75
C) P 0.80, R 0.60
D) P 0.60, R 0.60

5. Which stage gives object identity over time?

A) Detection
B) Tracking
C) Event logic
D) LLM summary

6. ByteTrack's key idea for keeping tracks during occlusion/blur?

A) Face recognition
B) Using low-score detections during association instead of discarding them
C) Optical flow only
D) An LLM

7. Dwell times are wrong — a person's timer keeps resetting. Most likely cause?

A) Track ID switches / lost tracks
B) Wrong video codec
C) LLM hallucination
D) Too many GPUs

8. The pipeline runs but no events fire on a busy clip. First thing to check?

A) ROI/line placement against the first frame
B) Swap YOLO for a bigger model
C) GPU drivers
D) LLM settings

9. What stops a crowd-threshold event from firing on every frame?

A) Confidence threshold
B) A cooldown interval
C) persist=True
D) Normalised coordinates

10. What is the correct role of the LLM in the IVA demo?

A) Watch the video and detect incidents
B) Summarise structured events only, adding no new facts
C) Identify people
D) Override event logic

11. How is a virtual-line crossing detected?

A) Box area exceeds threshold
B) Segment from previous centre to current centre intersects the line
C) Track ID changes
D) Confidence drops

12. Which metric belongs to the TRACKING evaluation level?

A) mAP
B) ID switches
C) Wall FPS
D) Hallucination rate

13. (2022) The DeepStream reference application is based on:

A) Mstream
B) GStreamer
C) Xstream
D) Lstream

14. (2022) Which GStreamer plugin sends telemetry to the cloud?

A) Gst-nvmsgconv
B) Gst-nvmsgbroker
C) Gst-nvinferserver
D) Gst-nvtracker

15. (2022) Stages of object tracking?

A) Designation & Motion
B) Motion & Detection
C) Designation, Motion, Recall
D) Recall & Motion

Week 11 — Answer key

1. **A) (1056, 270)** — x = 0.55 × 1920 = 1056; y = 0.25 × 1080 = 270. Normalised coords work at any resolution.
2. **C) 22.4 s** — dwell = now − 12.4 ≥ 10 → t ≥ 22.4 s.
3. **B) ~30 FPS** — Total 33 ms → ~30 FPS. Inference alone suggests ~66 FPS — why the demo reports wall FPS too.
4. **B) P 0.60, R 0.75** — Precision = 6/10; recall = 6/8.
5. **B) Tracking** — A detector has no memory across frames. Track ID ≠ real-world identity.
6. **B) Using low-score detections during association instead of discarding them** — Called via model.track(frame, persist=True, tracker='bytetrack.yaml'); persist keeps state across frames.
7. **A) Track ID switches / lost tracks** — Errors propagate from perception to semantics.
8. **A) ROI/line placement against the first frame** — Inspect the first frame (inspect_video.py), adjust config.yaml, or lower loiter_seconds.
9. **B) A cooldown interval** — Crowd = count of person tracks in ROI ≥ K, then cooldown.
10. **B) Summarise structured events only, adding no new facts** — Deterministic report first; LLM is a presentation layer.
11. **B) Segment from previous centre to current centre intersects the line** — Direction can be added by checking which side the track came from.
12. **B) ID switches** — Detection: precision/recall/mAP. Event: event F1. System: latency/throughput.
13. **B) GStreamer** — Pipelines built with GST-Python.
14. **B) Gst-nvmsgbroker** — Gst-nvmultistreamtiler forms the 2D frame array; NVIDIA Metropolis is the edge-to-cloud framework.
15. **C) Designation, Motion, Recall** — Challenges: speed, background distractions, viewpoint variation, occlusion — all of the above.

Week 12 — Healthcare AI

Clinical metrics math, calibration, Dice, CURB-65, governance, plus 2022 federated learning

1. A sepsis model: TP 80, FN 20, FP 40, TN 860. Sensitivity and PPV?

A) Sens 0.80, PPV 0.67
B) Sens 0.67, PPV 0.80
C) Sens 0.96, PPV 0.80
D) Sens 0.80, PPV 0.96

2. 200 patients were given a predicted risk of 30%; 60 had the outcome. The model at this level is:

A) Well calibrated
B) Over-predicting
C) Under-predicting
D) Can't tell

3. Segmentation: ground-truth mask 100 px, prediction 80 px, overlap 60 px. Dice score?

A) 0.50
B) 0.60
C) 0.67
D) 0.75

4. CURB-65: age 70, respiratory rate 32, BP 120/80, no confusion, urea normal. Score?

A) 0
B) 1
C) 2
D) 3

5. Why does the demo use an explicit CURB-65 calculator instead of asking the LLM for a severity score?

A) Faster
B) Inputs are shown and verifiable; an LLM could invent the score
C) Regulators ban LLMs
D) Saves GPU memory

6. A model works at the training hospital but performance drops at a new hospital with a different scanner. What step would have caught this?

A) Internal test split
B) External validation
C) More epochs
D) Heatmaps

7. The model runs in the background and logs predictions, but care decisions don't change. This is:

A) Canary
B) Silent deployment
C) Shadow IT
D) Federated learning

8. A hospital installs a new CT scanner; input image statistics shift. This is:

A) Label drift
B) Data drift
C) Calibration
D) Alert fatigue

9. Most important design choice for a clinical agent?

A) Largest possible model
B) Capability limitation: read-only, no autonomous diagnosis/orders, human approval
C) Autonomous chart editing
D) No logging

10. What decides whether clinical software may be regulated as a medical device?

A) Programming language
B) Intended use and claims
C) Model size
D) Cloud vs on-prem

11. FHIR stands for:

A) Federated Health Information Registry
B) Fast Healthcare Interoperability Resources
C) Federal Hospital Imaging Record
D) Fast Health Inference Runtime

12. Clinicians ignore a sepsis alert because it fires too often and isn't actionable. This is:

A) Alert fatigue — a workflow/threshold problem
B) Data drift
C) Overfitting
D) PHI leakage

13. (2022) Correct scatter-gather order: 1 workers validate & train, 2 server broadcasts, 3 workers track local best, 4 workers send updates, 5 server aggregates?

A) 3,2,1,5,4
B) 2,1,3,4,5
C) 1,2,3,4,5
D) 2,3,5,4,1

14. (2022) Homomorphic encryption lets you compute on data while it is:

A) Decrypted
B) Encrypted
C) Lost
D) Saved

15. (2022) Which statement about federated learning is FALSE? A: trains on heterogeneous data. B: trains a single model on multiple servers. C: relies on less powerful communication media.

A) Only B
B) Only C
C) B and C
D) A and C

Week 12 — Answer key

1. **A) Sens 0.80, PPV 0.67** — Sensitivity = 80/100 = 0.80. PPV = 80/120 = 0.667. (Specificity = 860/900 ≈ 0.956.)
2. **A) Well calibrated** — 60/200 = 30% observed ≈ 30% predicted.
3. **C) 0.67** — Dice = 2 × 60 / (100 + 80) = 0.667. (IoU = 60/120 = 0.5.)
4. **C) 2** — 1 point each: Confusion, Urea, Respiratory rate ≥30, low BP, age ≥65. Here RR + age = 2.
5. **B) Inputs are shown and verifiable; an LLM could invent the score** — Missing calculator inputs are marked unknown, never inferred silently.
6. **B) External validation** — External validation tests other sites, scanners, populations and time periods.
7. **B) Silent deployment** — Measures real-world performance, alert volume, latency, drift.
8. **B) Data drift** — Label drift = outcome meaning changes; performance drift = metrics decline. Retrained models need revalidation and approval.
9. **B) Capability limitation: read-only, no autonomous diagnosis/orders, human approval** — "Generate drafts, not decisions" + audit log.
10. **B) Intended use and claims** — "Draft for clinician review" differs from "AI diagnoses pneumonia".
11. **B) Fast Healthcare Interoperability Resources** — HL7 standard: Patient, Observation, Condition, MedicationRequest, Encounter, DiagnosticReport.
12. **A) Alert fatigue — a workflow/threshold problem** — Judge models by real decisions, not only AUROC.
13. **B) 2,1,3,4,5** — Broadcast → train → personalize → send → aggregate.
14. **B) Encrypted** — 2022 set: Clara (healthcare framework), NVFLARE = FL SDK, Admin API, gRPC protocol.
15. **A) Only B** — 2022 accepted answer: Only B.